

A Practitioner's Handbook of Information Retrieval

J. L. Clements

Contents

Chapter 1. Preface	1
Chapter 2. Foundations of information retrieval	3
2.1. The Boolean model	3
2.2. The vector-space model	3
2.3. The probabilistic model and BM25	4
Chapter 3. Index construction and query processing	5
3.1. The inverted index data structure	5
3.2. Offline construction	5
3.3. On-line query processing	6
3.4. Posting-list compression in detail	6
3.5. Skip lists and posting-list intersection	7
Chapter 4. Three case studies from production	9
4.1. Case study A: the recall collapse	9
4.2. Case study B: the head-tail tension	10
4.3. Case study C: the duplicate document outbreak	10
4.4. Case study D: the long-query degradation	11
4.5. A worked example: scoring a single query	11
Chapter 5. Advanced topics	13
5.1. Learning-to-rank	13
5.2. Neural retrieval	13
5.3. Cross-language and multilingual retrieval	14
5.4. Online evaluation	14
5.5. Counterfactual evaluation	15
5.6. Latency and throughput	15
Chapter 6. Appendix A: Glossary	17
Chapter 7. References	19
Chapter 8. Index	21

Chapter 1. Preface

This handbook grew out of a graduate seminar on information retrieval that the author taught between 2021 and 2025. The intended reader is an engineer or scientist who has already met the fundamentals of search engines once — in a textbook chapter, perhaps, or in an introductory course — and who now needs a single document covering the engineering details that the textbooks gloss over. Where standard references treat tf-idf in a paragraph and dispose of BM25 in a half-page derivation, we have tried to spend several pages on each: not because the mathematics is hard, but because the choices implicit in any production deployment (which normalisation? which document length statistic? which idf smoothing?) are exactly where deployments go wrong.¹

working title *Notes from the Search Engine Trenches*. The chapter on learning-to-rank in that draft was substantially shorter; the present edition expands it in light of the production experience described in chapter 3.

The book is roughly self-contained. Readers familiar with the standard literature [1] [2] [3] can skip the introductory material in chapter 1 and pick up at chapter 2; readers new to the field should read the chapters in order. Chapter 3 collects three case studies that, in our experience, capture the failure modes most often encountered in practice. The two appendices — a glossary of terminology and an annotated bibliography — are meant for reference rather than sequential reading.

A word on notation. We write document collections as $D = \{d_1, d_2, \dots, d_N\}$, individual documents as d , queries as q , and terms (the indexed unit, usually a stemmed word) as t . The function $tf(t, d)$ denotes the frequency of term t in document d , and $df(t)$ denotes the document frequency of t across D . All other notation is introduced where first used.

The author thanks the cohort of students who suffered through the seminar's first iteration, and the engineers at three search teams (unnamed, by their preference) whose production data informed the case studies in chapter 3. Errors and omissions are, of course, entirely the author's.

¹An earlier draft circulated in 2023 under the

Chapter 2. Foundations of information retrieval

Information retrieval (IR) is the practice of returning, from a collection of documents, the subset that best matches a stated information need. The need is expressed as a *query*; the matching function is called a *retrieval model*; the collection plus its auxiliary structures is called an *index*. This chapter develops the three classical retrieval models — Boolean, vector-space, and probabilistic — and discusses the engineering choices each model imposes on the index.[2]¹

built for legal and medical literature; the first textbook usage of the phrase "information retrieval" is generally credited to Calvin Mooers in 1950.

2.1. The Boolean model

The simplest retrieval model treats each document as a set of terms and each query as a Boolean formula over terms. The query $q = t_1 \wedge t_2 \vee t_3$ retrieves every document whose term-set contains either t_3 or both t_1 and t_2 . The model is sound, complete under its own semantics, and easy to implement on top of an inverted index.

It is also rarely the right model. Boolean queries are difficult for end-users to write,[3]² the result set is unranked (the system has no notion of *better* and *worse* matches), and the model has no mechanism for partial matches. Modern search engines retain a Boolean substrate — it is the cheapest way to compute the candidate set — but score the candidates with a ranked-retrieval model layered on top.

that fewer than 5% of queries contain any explicit Boolean operator. Even those that do are often used incorrectly: the operator AND is interpreted by many users as an English conjunction rather than a logical one, and queries like `books AND about AND cats` perform worse than the plain text `books about cats` would.

A short historical aside: Boolean retrieval was the operating mode of most production search systems through the 1980s, including the legal databases (LexisNexis, Westlaw) that dominated commercial IR until the web search era. The Boolean substrate is also the operating mode of every database engine's full-text search extension — PostgreSQL's `tsvector`, MySQL's `MATCH AGAINST`, SQLite's FTS5. The popularity of those engines is itself a testament to the model's continuing relevance for domains where exact match matters more than relevance ranking.[4]³

model often retain a Boolean filter layer for the same reason — to bound the candidate set before scoring. The combination of a Boolean filter with a ranked scorer is sometimes called *filtered retrieval* and is the default mode of operation in Lucene, the open-source library underlying Elasticsearch and Solr.

2.2. The vector-space model

The vector-space model, due to Salton [4], represents each document and each query as a vector in a V -dimensional space, where V is the vocabulary size. The components of the document vector $\vec{d}$ are weights assigned to each term in the vocabulary; the canonical weighting is *tf-idf*:

$$w(t, d) = tf(t, d) \cdot \log \frac{N}{df(t)}, \tag{1.1}$$

where $N = |D|$ is the size of the collection. The intuition is that a term contributes to a document's profile in proportion to how often it occurs in the document (the *tf* factor) and in inverse proportion to how widely it occurs in the collection (the *idf* factor). A term that appears in every document has $idf = 0$

¹The earliest IR systems date from the 1950s and were

²Studies of end-user query logs consistently show

³Conversely, deployments that adopt a ranked-retrieval

and contributes nothing.

Similarity between a query and a document is measured by the cosine of the angle between their vectors:

$$\text{sim}(q, d) = \frac{\text{vec}\{q\} \cdot \text{vec}\{d\}}{\|\text{vec}\{q\}\| \cdot \|\text{vec}\{d\}\|}. \tag{1.2}$$

The cosine normalisation is what makes the vector-space model work in practice. Without it, longer documents would score higher merely by virtue of having more terms; the cosine divides out document length and leaves a quantity that depends only on the *direction* of the vector in term-space, not its magnitude.

The vector-space model has two well-known weaknesses. First, the choice of weighting function is ad hoc; tf-idf works well in practice, but other weightings (log-tf, augmented-tf, pivoted length normalisation) work better on particular collections. Second, the model assumes that terms are independent, which is patently false — the words *car* and *automobile* are not independent, nor are *New* and *York*. The independence assumption is shared by the probabilistic models of the next section.

Practitioner’s note. The dot product $\text{vec } q \cdot \text{vec } d$ can be computed by walking the intersection of the two vectors’ non-zero components, which on a sparse representation is $O(\min(\text{vec } q_{\text{sub } 0}, \text{vec } d_{\text{sub } 0}))$. This is the operation that an inverted index is designed to make fast.

2.3. The probabilistic model and BM25

The probabilistic retrieval framework, due to Robertson and Sparck Jones [5], scores documents by the log-odds of relevance given the query. After several rounds of refinement [6] the framework crystallised into the *BM25* scoring function:

$$\text{BM25}(q, d) = \sum_{t \in q} \log \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} \cdot \frac{1}{k_1 + 1} \cdot \text{tf}(t, d) \cdot \left(1 - b + b \cdot \frac{|d|}{\overline{|d|}}\right) + \text{tf}(t, d), \tag{1.3}$$

where k_1 and b are tuning parameters (typically $k_1 \in [1.2, 2.0]$ and $b = 0.75$), $|d|$ is the length of document d , and $\overline{|d|}$ is the average document length. The numerator of the second factor saturates as tf grows large — a term occurring twenty times in a document is not twice as relevant as one occurring ten times — and the denominator penalises documents whose length exceeds the collection average.

BM25 remains, twenty-five years after its publication, the strongest single-feature ranker on most text collections. Modern learning-to-rank systems use it as one of dozens of features rather than as a stand-alone scorer, but its presence in the feature vector is near-universal.[5]¹ We note in passing that two of its variants merit attention. *BM25F* extends the score to documents with multiple weighted fields (title, body, anchor text); each field contributes its own length-normalised tf , and the per-field weights are tuned on a relevance-judgement set. *BM25+* adds a lower-bound term-frequency saturation [7] that prevents very short documents from being unduly penalised by the length-normalisation factor. Both variants live in the feature vectors of mature production rankers; neither displaces the original BM25 entirely.

variants developed at City University London in the early 1990s; BM1 through BM24 are now largely of historical interest. A unified derivation appears in [8].

¹The “25” in BM25 refers to its rank in a sequence of

Chapter 3. Index construction and query processing

Every retrieval model in chapter 1 reduces, at run time, to walking posting lists in an *inverted index*: a data structure that maps each term to the list of documents containing it. This chapter develops the index data structure, sketches the offline construction pipeline, and walks through the on-line query processing path.

3.1. The inverted index data structure

An inverted index is a key-value store keyed by *term* and valued by *posting list*. A posting list is a sequence of postings; each posting is a tuple containing at minimum a document identifier and typically also term frequency, term positions within the document, and any per-posting payload (field markers, payloads for proximity scoring, etc.).^[6]

historical accident. The earliest IR systems indexed each document as a forward list of terms; reversing the direction to go from term to documents gave the data structure its name. The forward index is still useful for some operations (snippet extraction, document scoring with stored term positions); a mature engine usually maintains both.

The figure below sketches the term-dictionary plus posting-list arrangement schematically; the dictionary maps each term to the offset and length of its posting list within the on-disk index.
[inline SVG omitted in non-SVG back-end]

The naive representation — a Python dict mapping strings to lists of integers — is conceptually correct but uses an order of magnitude more space than is necessary. Real implementations apply three layers of compression:

1. *Document IDs are gap-encoded.* The posting list for a common term might contain document IDs in the millions; storing the gaps between consecutive IDs typically halves the bit width per posting.
2. *Gaps are variable-byte or Elias-Fano encoded.* A gap of g is encoded in $\lceil \log_2 g \rceil$ bits rather than 32 or 64.
3. *Posting lists are paged.* The index lives on disk in 4 KiB or 16 KiB pages; queries memory-map the pages of interest rather than loading the entire index into RAM.

The result is an index that, for typical English collections, occupies roughly 20-30% of the size of the source documents in plain text [1]. Specialised compression schemes [9] can push this lower at the cost of decompression speed.

3.2. Offline construction

Building an inverted index from a collection of N documents is a batch job that proceeds in three phases:

1. **Tokenisation.** Each document is parsed into a sequence of terms. The tokeniser is responsible for sentence boundaries, case-folding, accent stripping, and stop-word removal. Stemming (reducing *running*, *runs*, *ran* to a common stem) is a policy decision: the Porter stemmer [10] is the traditional default, but lemmatisation or no stemming at all may be preferable for particular collections.

¹The name "inverted" is a (slightly unfortunate)

2. **Sort-and-merge.** Each document emits a sequence of (term, doc-id, position) triples; these are sorted by term, then by doc-id, then by position; runs of consecutive triples with the same term form the posting list for that term. On large collections the sort spills to disk in classical external-sort fashion.
3. **Compression and write-out.** The sorted runs are merged, compressed in the encoding chosen above, and written to the on-disk index. An auxiliary *term dictionary* maps each term to the offset of its posting list within the index file; the dictionary itself is small enough to keep in memory, or to load on demand via a B-tree on disk.

The whole pipeline scales linearly with the size of the input. A single machine can index roughly 10 GB of text per hour on modern hardware; distributing the work across a cluster — MapReduce-style [11] — pushes that to TB/hr.

3.3. On-line query processing

A query arrives as a string. The query processor walks four phases:

"query string -> tokens -> term ids -> posting walk -> ranked results"

The *tokens-to-term-ids* phase looks each token up in the term dictionary and discards tokens that don't appear in the collection. The *posting walk* phase opens the posting list for each remaining term, walks the lists in parallel (one cursor per term), and computes a score for every document that appears in at least one list. The walk can be either *term-at-a-time* (TAAT) or *document-at-a-time* (DAAT); DAAT permits early termination via the WAND algorithm [12] and is the default in modern engines.

The naive DAAT walk computes a full score for every candidate document, then sorts the candidates by score and returns the top k . For top- k queries this is wasteful: the system only needs the top k scores, not the complete ranked list. WAND maintains a running threshold equal to the score of the current k -th best document and short-circuits the walk whenever no remaining document can exceed that threshold. On typical collections WAND skips 90% or more of the candidate set, with no loss of accuracy.

Theorem 3.1: Let θ be the current threshold maintained by the WAND algorithm and let $U_{\text{sub } d}$ be an upper bound on the score any document remaining at posting cursor d can attain. If $U_{\text{sub } d} \leq \theta$, document d is guaranteed not to enter the top- k , and the algorithm may skip it.

Proof: The bound $U_{\text{sub } d}$ is computed from term-level score upper bounds $U_{\text{sub } t}$, which by construction satisfy $U_{\text{sub } t} \geq \text{score}(t, d)$ for every d in the posting list of t . Summing over the query terms gives the document bound. If this bound is at most the current top- k threshold θ , then no scoring of d can produce a value that displaces any of the current top- k documents, hence d may be skipped without affecting the final result set.

The threshold θ is initialised to zero and tightened as each new candidate enters the top- k heap. A loose initial threshold forces the algorithm to score the first k candidates fully; the investment is amortised over the cohort of subsequent candidates skipped via the bound.

3.4. Posting-list compression in detail

We give one concrete encoding to fix ideas. *Variable-byte* encoding represents an integer as a sequence of 7-bit groups, each group prefixed by a continuation bit: 0 in the final group, 1 in the non-final groups. An integer in $[0, 128)$ occupies one byte; an integer in $[128, 16384)$ occupies two bytes; and so on. The encoding is byte-aligned (no bit shuffling within a byte), which makes it fast to decode on commodity hardware.

Integer	Bytes (binary)	Bytes (hex)
0	00000000	00

127	01111111	7F
128	10000001 00000000	81 00
300	10000010 00101100	82 2C
16383	11111111 01111111	FF 7F
16384	10000001 10000000 00000000	81 80 00

Table 3.1.

Table 3.1 shows the encoding of representative integers. On typical English collections, gap-encoded variable-byte uses approximately 8-10 bits per posting — a compression ratio of 3-4 relative to the naive 32-bit-per-doc-id encoding.

A short worked decoding: the byte stream 81 80 00 82 2C decodes to the integers 16384 and 300 in that order. The decoder reads bytes until it sees one with the high bit clear; that byte is the last of the current integer. The remaining seven bits per byte are concatenated in big-endian order to recover the integer’s binary representation.

Variable-byte is byte-oriented, simple to decode, and within roughly a factor of two of the information-theoretic lower bound for our gap distributions. Tighter encodings exist — Golomb-Rice [9], PForDelta, Elias-Fano — but typically improve compression by only 10-20% at a substantial decoding-cost premium. The trade-off has historically favoured the simpler byte-aligned schemes; the recent re-introduction of SIMD-friendly bit-packed formats has begun to shift the balance.

3.5. Skip lists and posting-list intersection

Boolean conjunctive queries reduce to *posting-list intersection*: given lists $L_1, L_2, \dots, L_k$ (one per term), compute the sorted list of document IDs that appear in every L_i . The naive algorithm walks all k cursors in lock-step, advancing the cursor with the smallest doc-id at each step; total cost is the sum of the list lengths.

A *skip list* augmentation lets the algorithm jump forward in each list by more than one position when a much larger doc-id has been seen in another list. The skip pointers are emitted at regular intervals during posting-list compression (one skip per $\sqrt{|L|}$ postings is the textbook choice); the intersection walk falls back to the dense list only when the skip pointer would overshoot the current target.

For very short queries (two or three terms) the speed-up is modest, since the lists are walked sequentially in any case. For longer conjunctive queries against very common terms, where one list contains millions of postings and another only hundreds, skip lists turn a multi-second intersection into a millisecond one.^[7]

Moffat and Zobel’s work in the mid-1990s. The same data structure in another guise — William Pugh’s skip list as a balanced-tree replacement — is contemporaneous but unrelated; the IR variant uses fixed-stride skips while Pugh’s uses a randomised hierarchy.

¹The skip-list-as-IR-data-structure dates from

Chapter 4. Three case studies from production

The previous chapters developed the theory of information retrieval and the data structures it requires. This chapter switches register: we present three case studies, each drawn from a production search system, that illustrate the failure modes most commonly encountered in practice. Specific company and product names have been anonymised; details that would identify particular deployments have been altered or omitted.

4.1. Case study A: the recall collapse

A mid-sized commercial search engine, serving roughly 10 million queries per day, observed a sudden drop in user click-through rate on a Tuesday morning in 2023. Engineering response time was approximately four hours; the diagnosis took rather longer.

The system had several days earlier rolled out an upgrade to its stemmer. The new stemmer, which had been validated against an internal corpus of 50,000 queries, produced subtly different stems for compound words: where the old stemmer left *running-shoes* as *running-shoes*, the new one split it into *run + shoe*. The validation suite, which had been collected when *running-shoes* was a less common query, didn't surface the regression.

The fix was straightforward (revert the stemmer, then re-validate against a much larger and more recent query log) but the diagnostic process was instructive. Three observations:

1. *The query log is the validation set.* Any change to indexing policy must be validated against the actual distribution of queries, not a synthetic or older one. The recall regression would have been visible in a one-day click-through-rate experiment had one been run.
2. *Stemmer changes are unusually risky.* A stemmer change implicitly invalidates the entire index — documents that were previously findable under one stem may now be findable only under another. In retrospect the rollout should have been gated on a full re-index, not just a partial one.
3. *Click-through rate is a lagging indicator.* The drop was detected after roughly twelve hours of user-facing degradation. Synthetic monitoring (a small fixed set of queries replayed every few minutes) would have caught the regression in minutes.

The team's post-mortem recommended a triage protocol that has since been adopted as standard:

Step	Time	
Roll back changed component	< 5min	indent ctr
Replay synthetic monitor	< 5min	indent ctr
Diff old vs new index for top terms	< 1hr	indent ctr
Validate against query log sample	< 1day	indent ctr
Re-deploy with widened test	—	indent ctr

Table 4.1.

Table 4.1 summarises the protocol. The protocol is reactive rather than preventative; the deeper lesson, which the team has been less successful at internalising, is that *stemmer changes are schema*

changes and warrant the same engineering rigour.

4.2. Case study B: the head-tail tension

A vertical search engine in the legal-research domain found that optimising its ranker for top-of-result-list accuracy on common queries dragged down performance on rare ones. This *head-tail tension* is a recurring theme in production IR: ranking signals that work for queries seen thousands of times per day (where there is abundant click-through data) work badly for queries seen once or twice (where there is essentially no learned signal).

The team's solution was to maintain two ranker variants. The *head ranker*, trained on the top 1% of queries by frequency, used a hundreds-of-features gradient-boosted tree. The *tail ranker*, used for everything else, was a much simpler BM25-based model with a small number of hand-tuned features. A query-classification step decided at run time which ranker to use, with a default to the tail ranker on ambiguity.

The figure below, drawn from the post-deployment metrics report, shows the trade-off as a function of the head/tail cut-off.

[inline SVG omitted in non-SVG back-end]

The figure shows NDCG@10 on the head queries (upper curve) and the tail queries (lower curve) as a function of the percentile cut-off defining the head/tail boundary. The two curves intersect at approximately the 3% cut-off; below that, head queries dominate the training signal and tail queries suffer; above it, the head ranker loses its specialisation advantage [8]¹.

rank 10") is a standard IR metric that weights highly-ranked relevant results more heavily than lower-ranked ones. The discounted gain is normalised by the gain of the ideal ranking, yielding a metric in [0, 1].

The optimal operating point depends on the application. Web search, where the head distribution is very long-tailed, typically lives at the 1% cut-off or even lower. Legal-domain search, where the tail matters disproportionately, sits closer to 10%.

4.3. Case study C: the duplicate document outbreak

A consumer-facing search engine observed that its top-10 results for many queries contained two, three, or even five near-duplicate documents — different URLs serving essentially identical content, typically because the underlying CMS produced multiple HTML representations of the same article. The diversity of the result list was visibly degraded; users complained.

The fix was a deduplication pass, applied at index time, that clusters near-duplicate documents and retains only the canonical representative per cluster. The clustering used MinHash signatures [13] as the similarity primitive:

$$J(d_1, d_2) = \frac{|S(d_1) \cap S(d_2)|}{|S(d_1) \cup S(d_2)|}, \tag{3.1}$$

where $S(d)$ is the set of shingles (n -grams of n consecutive tokens, typically $n = 5$) extracted from d . The exact Jaccard similarity in (3.1) is expensive to compute over a collection of N documents (it is $O(N^2)$ in the worst case), so MinHash approximates J via the fraction of agreeing minimum-hash values across a fixed number of hash functions. With 128 hash functions, the approximation is within ± 0.04 of the true Jaccard with high probability.

Why shingles, not bag of words? A bag-of-words Jaccard treats word order as irrelevant. Two articles using identical vocabulary in different orders — a common pattern in plagiarism or template spam — collapse to a single canonical representative. Shingling preserves enough local order that genuine near-duplicates cluster while distinct articles do not.

¹NDCG@10 ("normalised discounted cumulative gain at

After deduplication the median number of distinct domains in the top 10 results rose from 5.8 to 7.4, and search-result diversity metrics improved across all measured query categories.

4.4. Case study D: the long-query degradation

A fourth, briefer case study: a question-answering search overlay on a help-desk knowledge base observed that user queries longer than roughly ten words performed substantially worse than shorter ones. The mean precision on three-word queries was 0.82; on ten-plus-word queries it dropped to 0.51.

The diagnosis took a week and uncovered a familiar problem in disguise. The first-pass ranker scored documents using a sum of per-term BM25 contributions, capped at twenty terms per query. Queries longer than twenty terms had their tail dropped — and the tail typically carried the *most discriminative* terms, since users who write long queries tend to include rare jargon at the end.

The fix was twofold. The first was to keep all query terms (the cap existed for ancient performance reasons that no longer held under modern hardware). The second was to reweight long-query scores by term IDF, so that rare specifying terms received more weight than common helper words. Mean precision on long queries rose to 0.77, and the gap to short queries narrowed substantially.

The case study illustrates a general rule: when a system's failure modes cluster in a particular query class, the cause is almost always a hard-coded threshold or cap that was tuned to a now-stale distribution of inputs. The fix is rarely a new algorithm; the fix is removing the limit and reweighting accordingly.

4.5. A worked example: scoring a single query

To make the chapter's machinery concrete we work through a single query end-to-end. The query is `efficient inverted index construction` and the collection contains four documents:

- d_1: "Efficient methods for index construction in large collections" (12 terms)
- d_2: "Inverted index data structures: a survey" (8 terms)
- d_3: "On efficient text search using inverted indices" (10 terms)
- d_4: "An introduction to information retrieval" (6 terms)

After stop-word removal and stemming the query reduces to `{efficient, invert, index, construct}`, and the document term-sets reduce as shown in Table 4.2.

Doc	efficient	invert	index	construct	Length
d_1	1	0	1	1	12
d_2	0	1	1	0	8
d_3	1	1	1	0	10
d_4	0	0	0	0	6

Table 4.2.

Document d_4 has no overlap with the query and is dropped immediately. The remaining three documents are scored by BM25 (1.3) with $k_1 = 1.5$, $b = 0.75$, and the corpus-wide statistics $df(\text{efficient}) = 2$, $df(\text{invert}) = 2$, $df(\text{index}) = 3$, $df(\text{construct}) = 1$, $N = 4$, $\overline{|d|} = 9$. The resulting scores — which we leave to the reader as a calculation exercise — rank $d_3 > d_1 > d_2$, which matches an intuitive reading of the documents' relevance to the query.

Chapter 5. Advanced topics

The first three chapters covered the core machinery of an information-retrieval system. This chapter sketches three advanced topics that build on that machinery: learning-to-rank, neural retrieval, and online evaluation.

5.1. Learning-to-rank

A *learning-to-rank* (LtR) system replaces the hand-tuned scoring function (BM25, tf-idf) with a model trained on labelled query-document pairs [14]. The labels are typically *graded relevance* (0 = irrelevant, 1 = marginally relevant, ..., 4 = perfectly relevant) and come from human annotators or from click-through data.

The training data consists of triples (q, d, y) where y is the relevance grade. The model learns a function $f(q, d; \theta)$ that maps query-document pairs to scores; training minimises a *ranking loss* that depends on the relative ordering of f values within a query, not on their absolute magnitudes. Three families of ranking losses dominate the literature:

1. *Pointwise* losses treat each (q, d, y) as an independent regression target: minimise $(f(q, d) - y)^2$. This is the simplest formulation and the easiest to train, but it ignores the ranking structure of the problem.
2. *Pairwise* losses operate on pairs (d^+, d^-) of documents with $y(d^+) > y(d^-)$: minimise $-\log(1 + \exp(f(q, d^-) - f(q, d^+)))$. RankNet [15] is the canonical example.
3. *Listwise* losses operate on entire result lists, directly optimising metrics like NDCG. LambdaMART [16] is the workhorse implementation.

Most production deployments use LambdaMART or a variant. The model is a gradient-boosted tree ensemble with hundreds of features per query-document pair; the features include BM25 scores, PageRank, click-derived signals, query-document text-overlap statistics, and many other signals specific to the deployment.

The features themselves are the major engineering effort. A list of representative features:

$f_1 = \text{BM25}(q, d)$, $f_2 = \text{BM25}(q, d_{\text{title}})$, $f_3 = \log |d|$, $f_4 = \text{age}(d)$, $\text{tag}\{4.1\}$

with additional features for query length, query class (navigational vs informational vs transactional), document language, and so on. A mature deployment will have several hundred features [17]; adding a new feature requires regenerating the entire training set, which is a substantial offline cost.

5.2. Neural retrieval

The 2018-2020 wave of pre-trained language models brought neural retrieval into the mainstream.^[9]

(2013), CDSSM (2014), and the early word-embedding approaches — showed promise but did not displace BM25 as the production default. The decisive step was the availability of large pre-trained transformers (BERT, 2018), which made it practical to obtain dense representations of comparable quality on much smaller fine-tuning datasets.

The architecture is summarised graphically in the figure below: the bi-encoder pre-computes document vectors at indexing time, while the cross-encoder defers all encoding work to query time.

¹Earlier work on neural retrieval — including DSSM

[inline SVG omitted in non-SVG back-end]

In the bi-encoder mode, similarity is computed by dot product in vector space [18]:

$$\text{score}(q, d) = \mathbf{E}_q(q) \cdot \mathbf{E}_d(d), \tag{4.2}$$

where $\mathbf{E}_q$ and $\mathbf{E}_d$ are the (possibly shared) encoder networks. The document encodings are pre-computed at index time; the query encoding is computed at query time; the similarity search reduces to a nearest-neighbour search in a high-dimensional vector space, for which efficient algorithms (HNSW [19], IVF-PQ) are well-developed.

Neural retrieval excels at queries whose lexical form differs from the desired documents — the *vocabulary mismatch* problem that classical IR has wrestled with for fifty years. A query "how do I fix a leaky tap" can match documents about "repairing dripping faucets" because the encoder maps both to nearby points in vector space, even though they share no tokens.

The classical and neural approaches are complementary. *Hybrid retrieval* runs both an inverted-index lookup (for lexical match) and a vector-space nearest-neighbour search (for semantic match), then combines the two ranked lists. In production, the cost is roughly twice that of a single retrieval path, but the recall improvement over either system alone is consistently large.

A second neural-retrieval architecture, the *cross-encoder*, takes the concatenated query-document pair as input to a single transformer and emits a relevance score directly. Cross-encoders are more accurate than bi-encoders — they can attend across query and document tokens jointly — but are quadratically more expensive, since the document encoding cannot be pre-computed. In production cross-encoders are used as a *re-ranker* applied to the top hundred candidates from a bi-encoder, where their cost is bounded.

The vector-search step in neural retrieval introduces its own engineering trade-offs. Exact nearest-neighbour search in a d -dimensional space over N documents takes $O(Nd)$ time, which is too slow for live serving when N is in the hundreds of millions. Approximate-nearest-neighbour structures sacrifice exact recall for sub-linear query time. HNSW, the de facto default, achieves recall-at-10 of 0.95 with a typical query cost equivalent to a few hundred distance computations per query; the index itself fits in memory at roughly $50N$ bytes (for 32-dim float vectors, $50 \times 32 \times 4 = 6400$ bytes per document). IVF-PQ is the preferred structure when memory is the binding constraint; it quantises each vector to a fixed number of bytes at modest recall loss.

5.3. Cross-language and multilingual retrieval

A topic worth a brief mention: retrieval across languages. Two distinct problems live under this heading. *Cross-language information retrieval* (CLIR) is the task of retrieving documents in language ℓ_d given a query in language ℓ_q . *Multilingual retrieval* is the task of returning relevant documents from a collection containing multiple languages, when the query language matches some subset of the documents.

CLIR was the harder problem in the pre-neural era: it required either machine-translation of the query into each document language (expensive, error-prone) or building a parallel cross-lingual representation through dictionary alignment. The neural-retrieval era simplified things substantially: a multilingual bi-encoder trained on parallel sentences learns a shared embedding space that collapses cross-lingual into single-language retrieval. The trade-off is that the embedding space is coarser than a monolingual one, so within-language recall typically drops a few percentage points.

Multilingual retrieval at indexing time is a question of analyser selection: each document is fed through the analyser appropriate to its language (Porter for English, GermanAnalyzer for German, etc). Document-language identification, done at indexing time on document text, is the gating step; modern n -gram language identifiers reach 98% accuracy on documents of 50 words or more.

5.4. Online evaluation

Offline metrics (NDCG, MAP, MRR) measure ranking quality on labelled data; *online* metrics measure user behaviour on live traffic. The two correlate imperfectly: a change that improves offline

NDCG can fail in production for any number of reasons — the offline label distribution may not match the live distribution, the metric may not weight user-visible improvements correctly, the system may have second-order effects (latency, UX) the offline metric ignores.

The standard online evaluation methodology is the *A/B test*: a fraction of live traffic is served by the experimental system, the rest by the control. After a sufficient sample is collected, the two cohorts are compared on a primary success metric (typically a click-through rate variant, or a downstream business metric) and on guard-rail metrics that protect against regressions in latency, crash rate, or session abandonment.

Two pitfalls deserve mention. First, *novelty effects*: users may behave differently with a new system simply because it is new, and the difference may decay over weeks. A/B tests must run long enough to outlast the novelty period. Second, *Simpson's paradox*: an experimental system can improve a metric within every traffic segment yet degrade the metric in aggregate, if the segment-mix shifts between the cohorts. A/B tests must therefore report results per-segment, not only in aggregate.

A more subtle pitfall is *interference between concurrent experiments*. Real deployments run dozens or hundreds of A/B tests simultaneously, and the experimental cohorts are not always disjoint. If two experiments interact — because they touch overlapping code paths, or because they jointly modify a user-facing surface — the measured effects of each can be biased by the other. Disciplined deployments stratify cohorts to keep interacting experiments on disjoint user samples, and re-run pre-launch experiments after major surface changes to confirm the original effect size still holds.

5.5. Counterfactual evaluation

A complementary technique to A/B testing, increasingly used in practice, is *counterfactual* (or *off-policy*) evaluation. Given a log of past system behaviour — queries, the results the live system returned, and the user's interactions with them — the counterfactual estimator infers what would have happened had a *different* ranker been in production. The estimator uses inverse-propensity-score weighting [10]¹ to correct for the bias introduced by the live ranker's preference ordering.

probability $p(a)$, an off-policy estimator weights each logged sample by $\pi(a) / p(a)$ where π is the candidate policy. The variance of the estimator grows with the inverse-propensity weights; deployments truncate weights at a fixed ceiling to trade bias for variance.

The technique is cheap (no live traffic required, no candidate-system deployment), unbiased under modest assumptions, and extremely sensitive to violations of those assumptions. Production practice is to use counterfactual estimates as a *filter* on candidate changes — discarding ideas that don't even look promising offline — and to gate launches on a live A/B test.

5.6. Latency and throughput

A topic the previous sections have understated: a search system that is correct but slow is a search system that is useless. Modern user expectations sit at roughly 200 ms for the rendered result page; the underlying retrieval-and-rank pipeline must deliver in under 50 ms to allow time for assembly and delivery. Achieving this on collections of 10^{10} documents requires a great deal of engineering attention.

The dominant costs are (1) the posting-list walk, (2) feature computation for the LtR ranker, and (3) the JVM or Go garbage collector. Cost (1) is amortised by aggressive use of WAND, by pre-filtering at the term-dictionary level, and by per-segment caching of top-k candidates for hot queries. Cost (2) is amortised by computing features lazily (only on candidates that survive the top-k heap threshold) and by hoisting expensive features to a re-ranker that runs on far fewer candidates than the first-pass ranker. Cost (3) is amortised by careful allocation discipline: posting-list cursors and feature vectors are pooled across queries, and short-lived allocations are avoided in the hot path.^[11]²

here. The points apply, mutatis mutandis, to Lucene/Java, to hand-written C++ engines, and to the Rust and Go engines that have appeared in the last few years.

¹Given a logging policy that selected action a with

²We avoid prescribing specific runtime systems

Chapter 6. Appendix A: Glossary

The following glossary covers the major terms used in the body of this handbook. Cross-references between glossary entries are not exhaustive; the reader is encouraged to use the index for the more complete picture.

Boolean retrieval. A retrieval model in which queries are Boolean formulae over terms and documents either match or do not match. See chapter 1.

BM25. A probabilistic retrieval scoring function widely used as the baseline ranker in modern systems. The acronym stands for "Best Matching 25". See section 1.3.

Bi-encoder. A neural retrieval architecture in which query and document are encoded independently into vectors whose dot product gives the relevance score. See section 4.2.

DAAT. Document-at-a-time query processing: posting lists are walked together, document by document, with full scoring of each candidate. Contrast with TAAT (term-at-a-time).

df. Document frequency: the number of documents in the collection containing a given term. The idf factor in tf-idf is $\log(N / df)$.

HNSW. Hierarchical Navigable Small World graph: an approximate-nearest-neighbour data structure used for the vector-search step in neural retrieval.

Inverted index. The fundamental IR data structure: a map from term to posting list.

Jaccard similarity. A set similarity metric defined as $|S_1 \cap S_2| / |S_1 \cup S_2|$. Used in near-duplicate detection via MinHash.

LambdaMART. A listwise gradient-boosted-tree learning-to-rank model. The de facto standard for production LtR.

MinHash. A randomised algorithm for estimating Jaccard similarity in time linear in the number of hash functions, regardless of set size. See section 3.3.

NDCG. Normalised discounted cumulative gain: a position-aware IR metric for graded-relevance evaluation.

Posting. A single entry in a posting list, containing at minimum a document identifier and typically additional payload (term frequency, positions).

Posting list. The list of postings for a given term in an inverted index.

Stemming. The reduction of morphological variants of a word to a common stem, e.g. *running*, *runs*, *ran* to *run*. See section 2.2.

tf. Term frequency: the number of times a given term appears in a given document.

tf-idf. A classical term-weighting scheme: $tf \cdot \log(N / df)$. See section 1.2.

WAND. Weighted And: an early-termination algorithm for top-k DAAT query processing.

Chapter 7. References

1. C. D. Manning, P. Raghavan, and H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, 2008. The standard undergraduate textbook.
2. W. B. Croft, D. Metzler, and T. Strohman. *Search Engines: Information Retrieval in Practice*. Pearson, Boston, 2010. A practitioner-oriented companion to Manning et al., stronger on engineering details.
3. C. Zhai and S. Massung. *Text Data Management and Analysis*. Morgan and Claypool, San Rafael, 2016. Broader in scope; covers analytics and NLP applications alongside core IR.
4. G. Salton, A. Wong, and C. S. Yang. A vector space model for automatic indexing. *Communications of the ACM* 18(11):613-620, 1975. The original vector-space-model paper.
5. S. E. Robertson and K. Sparck Jones. Relevance weighting of search terms. *Journal of the American Society for Information Science* 27(3):129-146, 1976. The foundational probabilistic-retrieval paper.
6. S. E. Robertson, S. Walker, S. Jones, M. M. Hancock-Beaulieu, and M. Gatford. Okapi at TREC-3. In *Proceedings of TREC-3*, 1995. Introduces BM25 in its current form.
7. Y. Lv and C. Zhai. Lower-bounding term frequency normalisation. In *CIKM 2011*. Introduces BM25+ and related corrections.
8. S. E. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval* 3(4):333-389, 2009. The modern unified derivation of the BM25 family.
9. I. H. Witten, A. Moffat, and T. C. Bell. *Managing Gigabytes*. Second edition. Morgan Kaufmann, San Francisco, 1999. The standard reference on index compression.
10. M. F. Porter. An algorithm for suffix stripping. *Program* 14(3):130-137, 1980. The classic Porter stemmer.
11. J. Dean and S. Ghemawat. MapReduce: Simplified data processing on large clusters. *Communications of the ACM* 51(1):107-113, 2008. The MapReduce paper.
12. A. Z. Broder, D. Carmel, M. Herscovici, A. Soffer, and J. Zien. Efficient query evaluation using a two-level retrieval process. In *CIKM 2003*. The WAND algorithm.
13. A. Z. Broder. On the resemblance and containment of documents. In *Proc. Compression and Complexity of Sequences*, 1997. The MinHash paper.
14. T.-Y. Liu. Learning to rank for information retrieval. *Foundations and Trends in Information Retrieval* 3(3):225-331, 2009. The canonical learning-to-rank survey.
15. C. Burges, T. Shaked, E. Renshaw, A. Lazier, M. Deeds, N. Hamilton, and G. Hullender. Learning to rank using gradient descent. In *ICML 2005*. RankNet.

16. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010. The LambdaMART exposition.
17. O. Chapelle and Y. Chang. Yahoo! learning to rank challenge overview. In *Proceedings of the Learning to Rank Challenge*, 2011. The Yahoo! LtR dataset paper.
18. V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih. Dense passage retrieval for open-domain question answering. In *EMNLP 2020*. The DPR paper.
19. Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42(4):824-836, 2018. The HNSW paper.

The works below were cited throughout the text. They are an annotated bibliography rather than a comprehensive one: each entry carries a one-sentence note on the work’s scope and its place in the literature.

Chapter 8. Index

-1p **BM25** 1 ; tuning parameters 1 ; saturation 1
Bi-encoder 4
Boolean retrieval 1
Click-through rate 3
DAAT 2 ; **WAND** 2 ; threshold 2
Deduplication 3 ; **MinHash** 3 ; shingling 3
Document frequency 1
HNSW 4
Inverted index 2 ; compression 2 ; construction 2
Jaccard similarity 3
LambdaMART 4
Learning-to-rank 4 ; pointwise 4 ; pairwise 4 ;
listwise 4
MinHash 3
NDCG 3
Neural retrieval 4 ; bi-encoder 4 ; hybrid 4
Online evaluation 4 ; A/B test 4 ; novelty 4
Porter stemmer 2
Posting list 2 ; compression 2 ; variable-byte 2
Probabilistic model 1
Query processing 2 ; **DAAT** 2 ; **TAAT** 2 ; **WAND** 2
Recall regression 3
Shingling 3
Stemming 2 ; **Porter** 2 ; risks 3
Stop-word removal 2
Term frequency 1
tf-idf 1
Variable-byte encoding 2
Vector-space model 1
WAND algorithm 2